Strategy Pattern

L'infrastruttura dei linguaggi

Nested class, inner class, local class
e i loro equivalenti

Java · C++ · Python · C

Appunti di Ingegneria del Software

*Il pattern in se' e' dato per noto: qui interessa **con quali meccanismi del linguaggio** lo si costruisce, e cosa cambia nel comportamento a seconda della forma scelta.*

*Ogni affermazione tecnica di questo documento e' stata **misurata** compilando ed eseguendo (g++ 15.2.1, gcc 15.2.1, JDK 21, Python 3.13):*
i sorgenti sono in `prove/strategy/`.

Anno 3

Contents

Chapter 1 — Il problema infrastrutturale — Page 3

1.1 Cosa serve, meccanicamente, per fare Strategy 3
1.2 Il punto di partenza: la frase-chiave delle slide 3
1.3 La mappa: quale forma esiste in quale linguaggio 4
1.4 L'esempio comune: `SimpleCalc` 4

Chapter 2 — Java: l'interfaccia come tipo Strategy — Page 5

2.1 Dichiarazione 5
2.2 Implementazione 6
2.3 Default e static method 6
2.4 Le due regole di risoluzione dei conflitti 6
Regola 1: l'istanza vince — 7 • Regola 2: i gia' sovrascritti sono ignorati — 7 • Il conflitto vero: due default indipendenti — 7 • Un metodo di istanza ereditato soddisfa un abstract — 8 • Il riepilogo delle slide — 8
2.5 Il listener: l'interfaccia a un solo metodo 8
2.6 Functional interface: il ponte verso le lambda 9

Chapter 3 — Java: quattro forme piu' la lambda — Page 10

3.1 Le tre categorie delle slide 10
3.2 La sintassi di creazione 11
3.3 Le tre classi dello Strategy in Java 11
Strategy e Context — 11 • Forma A — static nested — 11 • Forma B — inner class — 12 • Forma C — local class — 12 • Forma D — classe anonima — 13 • Forma E — lambda — 13
3.4 La misura decisiva: i nomi generati dal compilatore 13

Chapter 4 — C++: solo nested, piu' le lambda — Page 15

4.1 L'equivalente dell'interfaccia: la classe astratta pura 15
I tre casi delle slide, tradotti — 15
4.2 "In C++ this is just related to visibility and naming" 16
4.3 Le tre classi dello Strategy in C++ 16
Strategy e Context, con le strategie annidate — 16 • Forma B — l'equivalente dell'inner class, a mano — 17 • Forma C — local class, e i suoi due vincoli — 17 • Forma D — la lambda: la vera classe locale del C++ — 18

Chapter 5 — Python: un solo costrutto, tre comportamenti — Page 20

5.1 L'equivalente dell'interfaccia: tre possibilita' 20

5.2 I tre casi delle slide, risolti dall'MRO 20
5.3 Nested, inner, local: un solo costrutto 21
5.4 Misura 1: la classe annidata NON vede lo scope esterno 21
5.5 Misura 2: la classe locale SI' cattura (a differenza del C++) 22
5.6 Misura 3: ogni chiamata crea una classe NUOVA 22
5.7 Le tre classi dello Strategy in Python 22

Chapter 6 C: senza classi, senza scope, senza closure Page 24
6.1 L'equivalente dell'interfaccia: la struct di puntatori 24
6.2 Scoperta 1: annidare una `struct` NON crea uno scope 24
6.3 Scoperta 2: le funzioni locali non esistono in C standard 25
6.4 Le tre classi dello Strategy in C 25
Strategy: una struct di puntatori a funzione — 25 • Context e delega — 26 • Le tre forme, tradotte — 26 • Forma D — puntatore a funzione e contesto `void*` — 26

Chapter 7 Confronto finale Page 28
7.1 La stessa strategia, quattro infrastrutture 28
7.2 Il TIPO della strategia: le quattro dichiarazioni 28
7.3 Le quattro affermazioni da ricordare 28
7.4 Come scegliere, in pratica 29

Chapter 8 Esecuzione e verifica Page 30
8.1 I quattro programmi 30
8.2 Esperimenti: rompili apposta 30

Chapter 9 Fonti Page 32

Chapter 1

Il problema infrastrutturale

1.1 Cosa serve, meccanicamente, per fare Strategy

Il pattern richiede due sole capacita' dal linguaggio:

1. **memorizzare un riferimento a del codice** in una struttura dati;
2. **sostituirlo** con un altro riferimento.

Tutto il resto (interfacce, classi astratte, ereditarieta') e' un *modo* di ottenere queste due cose, non un requisito. I meccanismi disponibili sono quattro: **puntatori a funzione** nativi, **first-class function**, **classi o istanze di classi**, **reflection**.

Proposition 1.1.1 Perche' le slide del corso parlano di late binding

Le slide aprono con una motivazione che spiega tutto il resto:

> *"Here we reflect substantially on how the strategy pattern is achieved in Java. This is at the core of polymorphism via overriding and late binding.* ***Indeed, this is also because we do not use pointers to functions.****"*

Java non ha puntatori a funzione: deve ottenere lo stesso risultato con **oggetti** e **late binding**. E' per questo che le slide, prima di arrivare al pattern, dedicano trenta pagine a *nested class*, *inner class*, *local class*, *classi anonime* e *interfacce*: sono l'infrastruttura con cui, in Java, si impacchetta una strategia.

Fonte: slide L04*, pag. 4.*

1.2 Il punto di partenza: la frase-chiave delle slide

> *"In Java and in C++ it's possible to place a class definition within another class definition.* ***In C++ this is just related to visibility and naming. In Java the matter is deeper****; we can distinguish between* nested classes*,* inner classes *and* local classes*."*

Fonte: slide L04*, pag. 6.*

Questo documento prende quella frase e la verifica in tutti e quattro i linguaggi, misurando che cosa cambia davvero.

1.3 La mappa: quale forma esiste in quale linguaggio

	Java	**C++**	**Python**	**C**
classe annidata (namespacing)	si' (`static`)	si'	si'	**no**: il nome resta globale
legame implicito con l'istanza esterna	**si'** (inner class)	no	no	no
classe locale in una funzione	si'	si'	si'	solo la `struct`, senza metodi
la classe locale cattura le variabili locali	si' (*effectively final*)	**NO**	**si'** (closure)	no
classe anonima / senza nome	si'	lambda	`type()` o lambda	no
puntatori a funzione	no	si'	(funzioni first-class)	**si'**

Note:

Le due caselle piu' sorprendenti, entrambe verificate col compilatore:

- in **C++** una classe locale **non puo'** leggere le variabili automatiche della funzione che la contiene — cosa che in Java e Python si fa normalmente;
- in **C** annidare una `struct` **non crea alcuno scope**: il nome e' visibile anche fuori (in C++ la stessa riga e' un errore).

1.4 L'esempio comune: `SimpleCalc`

Per confrontare i quattro linguaggi a parita' di problema, tutti gli esempi implementano la stessa **calcolatrice** delle slide (`SimpleCalc`, pagg. 37–50), resa *headless* per poter girare senza display grafico:

- **Strategy** = `Operazione`, con un metodo `applica(a, b)`;
- **Context** = `Calcolatrice`, che tiene un accumulatore e l'operazione corrente, e vi *delega*;
- **ConcreteStrategy** = le singole operazioni, implementate ogni volta con una forma di classe diversa.

Il calcolo di prova e' sempre lo stesso: `20` ⟨`op`⟩ `6`, cosi' i risultati sono confrontabili a colpo d'occhio fra linguaggi.

Chapter 2

Java: l'interfaccia come tipo Strategy

Prima di scegliere *dove* mettere l'implementazione (nested, inner, local, anonima — capitolo 3), bisogna dichiarare il **tipo** della strategia. In Java quel tipo e' quasi sempre un'**interfaccia**, ed e' il costrutto a cui le slide dedicano piu' spazio (pagg. 13–36).

2.1 Dichiarazione

Definition 2.1.1: Interface (slide 13)

Una dichiarazione di interfaccia introduce un **nuovo reference type** i cui membri sono classi, interfacce, costanti e metodi. Questo tipo **non ha variabili di istanza** e tipicamente dichiara uno o piu' **metodi astratti**; classi altrimenti *non imparentate* possono implementarla fornendone le implementazioni.

Le interfacce **non possono essere istanziate**: possono solo essere implementate da classi o estese da altre interfacce. Contengono solo costanti, firme di metodi, default method, static method e tipi annidati.

Fonte: slide L04*, pag. 13.*

```
public interface Bicycle {
    void changeCadence(int newValue);
    void changeGear(int newValue);
    void speedUp(int increment);
    void applyBrakes(int decrement);
}
```

Fonte: slide L04*, pag. 14.*

Proposition 2.1.1 Perche' Java ammette l'ereditarieta' multipla di interfacce

Le slide danno la ragione precisa: le interfacce *"support single and multiple inheritance, since **the absence of instance data eliminates the ambiguities** of, say, C++"*.

Non avendo stato di istanza, non esiste il problema dei **dati duplicati** che in C++ richiede l'ereditarieta' virtuale. Resta solo il problema del *comportamento* duplicato, risolto dalle due regole della sezione 2.4.

Fonte: slide L04*, pag. 14.*

Example 2.1.1 (Verificato: un'interfaccia ha davvero solo costanti)

Interrogando a runtime i campi dichiarati in un'interfaccia con costanti:

```
campi dichiarati in un'interfaccia:
  E  ->  static=true final=true public=true
```

Tutte le costanti sono **implicitamente** `public static final`; tutti i metodi abstract, default e static sono **implicitamente** `public`. Nessuno stato di istanza, come promesso.

Fonte: slide L04, *pag. 15.*
Misurato in `prove/strategy/NL5_Java_Interfacce.java` *(compilato ed eseguito).*

2.2 Implementazione

```
class BianchiBicycle implements Bicycle {
  int cadence = 0, speed = 0, gear = 1;
  // Il compilatore RICHIEDE che changeCadence, changeGear, speedUp e
  // applyBrakes siano tutti implementati: la compilazione FALLISCE se
  // uno di essi manca.
  void changeCadence(int newValue) { cadence = newValue; }
  // ...
}
```

Fonte: slide L04, *pag. 16.*

2.3 Default e static method

Definition 2.3.1: Default e static method (slide 15, 29)

Un **default method** (parola chiave `default`) ha un corpo e **puo' essere sovrascritto**: e' il meccanismo che permette l'*evoluzione* di un'interfaccia, supportando l'implementazione non obbligatoria di tutti i metodi nelle classi.

Un **static method** serve per comportamenti generali dell'interfaccia: **non** puo' accedere a dati non-static e **non** puo' essere sovrascritto.

Fonte: slide L04, *pag. 29.*

Example 2.3.1 (Il problema che i default risolvono (esempio Oracle))

Data un'interfaccia gia' pubblicata e usata da altri:

```
public interface DoIt {
   void doSomething(int i, double x);
   int  doSomethingElse(String s);
}
```

volendo aggiungere un metodo, **tutto il codice delle vecchie implementazioni si romperebbe**, perche' non ha quel metodo. Con `default` il problema sparisce:

```
public interface DoIt {
   void doSomething(int i, double x);
   int  doSomethingElse(String s);
   default boolean didItWork(int i, double x, String s) {
       // Method body
   }
}
```

Verificato: una classe scritta *prima* dell'aggiunta di `didItWork` continua a compilare e a funzionare, ereditando il default.

Fonte: slide L04, *pag. 30-32.*
Misurato in `prove/strategy/NL5_Java_Interfacce.java` *(compilato ed eseguito).*

2.4 Le due regole di risoluzione dei conflitti

Default e abstract method si ereditano come metodi di istanza. Ma quando i supertipi forniscono **piu' default method con la stessa firma**, il compilatore applica due principi:

Proposition 2.4.1 Le due regole (slide 17)

1. i **metodi di istanza sono preferiti** ai default method delle interfacce;
2. i metodi **gia' sovrascritti da altri candidati sono ignorati** — circostanza che si presenta quando i supertipi condividono un antenato comune.

Fonte: slide L04, *pag. 17.*

2.4.1 Regola 1: l'istanza vince

```
public class Horse {
  public String identifyMyself() { return "I am a horse."; }
}
public interface Flyer {
  default public String identifyMyself() { return "I am able to fly."; }
}
public interface Mythical {
  default public String identifyMyself() {
      return "I am a mythical creature."; }
}
public class Pegasus extends Horse implements Flyer, Mythical { }
```

Output verificato: `Pegasus.identifyMyself()` restituisce *"I am a horse."* — il metodo di istanza ereditato dalla *classe* `Horse` vince sui default delle due interfacce.
Fonte: slide L04, *pag. 18-19.*

2.4.2 Regola 2: i gia' sovrascritti sono ignorati

```
public interface Animal {
    default public String identifyMyself() { return "I am an animal."; }
}
public interface EggLayer extends Animal {
    default public String identifyMyself() { return "I am able to lay eggs."; }
}
public interface FireBreather extends Animal { }      // NON fa override

public class Dragon implements EggLayer, FireBreather { }
```

Output verificato: *"I am able to lay eggs."* — `EggLayer` ha gia' sovrascritto il default di `Animal`, quindi quello di `Animal` (che `FireBreather` porterebbe) viene **ignorato**. Nessuna ambiguita', nessun override necessario.
Fonte: slide L04, *pag. 20-21.*

2.4.3 Il conflitto vero: due default indipendenti

Quando due default **indipendenti** confliggono (o un default confligge con un abstract), il compilatore **produce un errore** e si e' obbligati a fare l'override esplicito.

```
public interface OperateCar {
    default public int startEngine(EncryptedKey key) { /* impl */ }
}
public interface FlyCar {
    default public int startEngine(EncryptedKey key) { /* impl */ }
}
public class FlyingCar implements OperateCar, FlyCar { }   // ERRORE
```

Errore reale prodotto dal compilatore:

```
error: types OperateCar and FlyCar are incompatible;
  class FlyingCar inherits unrelated defaults for
  startEngine(EncryptedKey) from types OperateCar and FlyCar
```

La correzione, con la sintassi `Interfaccia.super.metodo()`:

```
public class FlyingCar implements OperateCar, FlyCar {
    public int startEngine(EncryptedKey key) {
        FlyCar.super.startEngine(key);
        OperateCar.super.startEngine(key);
    }
}
```

Note:

Il nome che precede `super` deve riferirsi a una **superinterfaccia diretta** che definisce o eredita un default per quel metodo. Questa forma di invocazione non e' limitata al caso del conflitto: `super` si puo' usare per invocare un default method sia in classi sia in interfacce.

Fonte: slide L04, *pag. 22-25.*
Misurato in ***prove/strategy/NL5_Java_Interfacce.java*** *(compilato ed eseguito).*

2.4.4 Un metodo di istanza ereditato soddisfa un abstract

```
public interface Mammal { String identifyMyself(); }   // ASTRATTO
public class Horse { public String identifyMyself() { return "I am a horse."; } }

public class Mustang extends Horse implements Mammal { }   // non implementa nulla!
```

Output verificato: *"I am a horse."* — `Mustang` eredita `identifyMyself()` da `Horse`, e quel metodo concreto **soddisfa** il contratto astratto di `Mammal`.
Fonte: slide L04, *pag. 26-27.*

2.4.5 Il riepilogo delle slide

	Metodo di istanza della superclasse	**Metodo static della superclasse**
Metodo di istanza della sottoclasse	*Overrides*	errore di compilazione
Metodo static della sottoclasse	errore di compilazione	*Hides*

Fonte: slide L04, *pag. 28.*

2.5 Il listener: l'interfaccia a un solo metodo

Le slide arrivano allo Strategy attraverso gli **event listener** di Swing. Un listener *e'* un'interfaccia con un solo metodo:

```
public interface ActionListener extends EventListener {
    public void actionPerformed(ActionEvent e);
}
```

Fonte: slide L04, *pag. 36.*

Proposition 2.5.1 ActionListener *e'* una Strategy

- l'**interfaccia** `ActionListener` e' il tipo Strategy: incapsula "cosa fare quando succede l'evento";
- il `JButton` e' il **Context**: non sa cosa fara' il listener, lo invoca e basta;
- `addActionListener()` e' l'**iniezione** della strategia.

Le slide notano anche il vantaggio pratico: piu' componenti grafici possono condividere **lo stesso** listener, se fanno la stessa cosa — che e' esattamente il riuso promesso dallo Strategy.

Fonte: slide L04*, pag. 34-35.*

2.6 Functional interface: il ponte verso le lambda

Proposition 2.6.1 Verificato: una lambda richiede UN SOLO metodo astratto

Contando a runtime i metodi astratti dichiarati:

```
MioActionListener    metodi astratti: 1  -> e' una FUNCTIONAL INTERFACE
Bicycle              metodi astratti: 4  -> NON e' functional
```

Provando a implementare `Bicycle` con una lambda:

```
error: incompatible types: Bicycle is not a functional interface
    multiple non-overriding abstract methods found in interface Bicycle
```

E' per questo che i **default method contano** anche qui: aggiungere default method a un'interfaccia **non** le fa perdere lo status di functional interface, perche' i default non sono astratti. Un'interfaccia puo' avere venti default method e restare implementabile con una lambda.

Misurato in `prove/strategy/NL5_Java_Interfacce.java` *(compilato ed eseguito).*

Chapter 3

Java: quattro forme piu' la lambda

3.1 Le tre categorie delle slide

Definition 3.1.1: Nested class

Una classe dentro un'altra, dichiarata `static`. Serve per **naming e packaging**: permette di *nascondere l'esistenza* di una classe al mondo esterno e di raggruppare classi che appartengono logicamente insieme. Il nome e' **locale alla classe che la racchiude**.

Vincolo: le dichiarazioni in una nested class possono usare **solo i membri** `static` della classe esterna.

Fonte: slide L04, *pag. 7.*

Definition 3.1.2: Inner class

Una classe dentro un'altra **senza** `static`. In Java "dichiarare semplicemente una classe dentro il corpo di un'altra, senza ulteriori qualificazioni, produce una inner class".

Ogni istanza ha una **enclosing instance** e puo' usarne i membri — *anche quelli di istanza*. Non puo' avere membri `static`, ne' lo stesso nome della classe che la racchiude.

Fonte: slide L04, *pag. 8, 11.*

Definition 3.1.3: Local class

Una classe dichiarata **dentro il corpo di una funzione**. Utile quando servono codice e strutture dati per compiti specifici dentro una funzione, senza renderli disponibili all'esterno: e' un mezzo per imporre **incapsulamento a livello di scope di funzione**. Il nome e' locale allo scope che la racchiude.

In Java le local class sono **a tutti gli effetti inner class**, non semplici nested: accedono ai membri della classe esterna, alle variabili locali `final` e ai parametri.

Fonte: slide L04, *pag. 12.*

Example 3.1.1 (L'esempio canonico delle slide)

```
public class X {
  int instanceVar;           // appartiene a un'istanza di X
  static int staticVar;      // appartiene alla classe X

  private class Inner {                    // INNER (non static)
    int y;
    Inner() { y = instanceVar + staticVar; } // puo' usare ENTRAMBI
  }

  private static class Nested {            // NESTED (static)
    int x;
```

```
    Nested() {
      x = staticVar;
      // x += instanceVar;   // Errore! nessuna istanza di X disponibile
    }
  }
}
```

La differenza sta in *una sola parola chiave*, ma cambia la natura della classe: `Nested` e' una classe come le altre, `Inner` non esiste senza un oggetto `X` che la ospiti.

Fonte: slide L04, *pag. 9.*

3.2 La sintassi di creazione

Proposition 3.2.1 `esterno.new Inner()`

Una nested class si istanzia come una classe normale, col nome qualificato. Una **inner** class richiede una sintassi speciale, perche' serve *un'istanza dell'esterno*:

```
// nested (static): il nome e' qualificato dalla CLASSE
Calcolatrice.Somma s = new Calcolatrice.Somma();

// inner: il nome e' qualificato da un'ISTANZA
NL1_Java_SimpleCalc esterno = new NL1_Java_SimpleCalc();
Operazione inner = esterno.new MoltiplicaPerCampo();
```

Provare a scrivere `new MoltiplicaPerCampo()` da un contesto `static` e' un errore di compilazione: non c'e' nessuna enclosing instance da agganciare.

3.3 Le tre classi dello Strategy in Java

3.3.1 Strategy e Context

```
// STRATEGY: un'interfaccia ANNIDATA (le interfacce annidate sono
// sempre implicitamente static)
interface Operazione {
    int applica(int a, int b);
    default String simbolo() { return "?"; }
}

// CONTEXT: tiene lo stato (l'accumulatore) e delega alla strategia
static class Calcolatrice {
    private int accumulatore = 0;
    private Operazione operazione;             // la STRATEGIA

    void setOperazione(Operazione op) { this.operazione = op; }

    void premiNumero(int n) {                  // i "bottoni" delle slide
        if (operazione == null) { accumulatore = n; return; }
        accumulatore = operazione.applica(accumulatore, n);  // <- delega
    }
    int display() { return accumulatore; }
}
```

3.3.2 Forma A — static nested

```
static int contatoreStatico = 0;        // membro STATIC dell'esterna
private int campoDiIstanza = 100;       // membro di ISTANZA

static class Somma implements Operazione {   // <- static: e' NESTED
```

```
    @Override public int applica(int a, int b) {
        contatoreStatico++;             // OK: membro static
        // campoDiIstanza;              // ERRORE: non c'e' un'istanza!
        return a + b;
    }
    @Override public String simbolo() { return "+"; }
}
```

Perche' questa forma: la strategia non ha bisogno di conoscere lo stato di un particolare oggetto `Calcolatrice`; `static` lo dichiara esplicitamente ed evita di tenere in vita l'oggetto esterno. E' la forma da preferire per default.

3.3.3 Forma B — inner class

```
class MoltiplicaPerCampo implements Operazione {   // NON static: INNER
    @Override public int applica(int a, int b) {
        // legge il campo di ISTANZA dell'oggetto esterno:
        return a * b % campoDiIstanza;
    }
    @Override public String simbolo() { return "*%" + campoDiIstanza; }
}
```

Perche' questa forma: e' la scelta delle slide per i bottoni `+` e `-` di `SimpleCalc`. Il commento della slide 43 e' esplicito: *"this also has the advantage of allowing the inner class access to the components defined within the outer class"*. La strategia deve leggere e scrivere `currentCalc` e `calcOperation` della calcolatrice: una inner class lo fa senza che si debba passare nulla.

Example 3.3.1 (La prova che il legame e' vivo (output reale))

Lo **stesso** oggetto-strategia, dopo aver modificato un campo dell'*oggetto esterno*:

```
campoDiIstanza=100 -> 20*6 % 100 = 20
campoDiIstanza=7   -> 20*6 % 7   = 1
```

E creando l'inner class da *un'altra* istanza esterna:

```
inner di un'ALTRA istanza (campo=1000): 120
```

Non si tratta di una copia del valore fatta alla costruzione: la strategia **legge lo stato dell'enclosing instance al momento della chiamata**. Due istanze esterne danno due "mondi" separati.

Misurato in `prove/strategy/NL1_Java_SimpleCalc.java` (compilato ed eseguito).

3.3.4 Forma C — local class

```
static Operazione creaSottrazioneLimitata(int minimo) {
    // 'minimo' e' effectively final: catturabile
    class SottrazioneLimitata implements Operazione {   // visibile SOLO qui
        @Override public int applica(int a, int b) {
            return Math.max(a - b, minimo);   // <- CATTURA 'minimo'
        }
        @Override public String simbolo() { return "-(>=" + minimo + ")"; }
    }
    return new SottrazioneLimitata();
    // fuori dal metodo il NOME 'SottrazioneLimitata' non esiste piu'
}
```

Perche' questa forma: la classe serve solo a costruire *quel* particolare oggetto, e non ha senso come tipo pubblico. Il nome resta confinato al metodo (incapsulamento a livello di funzione), ma il *tipo* continua a esistere: l'oggetto restituito e' perfettamente utilizzabile fuori, attraverso l'interfaccia.

Proposition 3.3.1 La regola della cattura: *effectively final*

Una local class (e una classe anonima) puo' catturare una variabile locale solo se questa non viene mai riassegnata dopo l'inizializzazione. Il motivo e' che la cattura avviene **per valore**: il compilatore copia il valore nell'oggetto, e permettere riassegnazioni successive creerebbe due copie divergenti.

Nota la differenza rispetto ai *campi*: la forma B legge `campoDiIstanza` *ogni volta*, e infatti lo vede cambiare; la forma C ha una copia di `minimo` congelata alla creazione.

3.3.5 Forma D — classe anonima

```
static Operazione divisioneSicura() {
    return new Operazione() {              // <- classe SENZA nome
        @Override public int applica(int a, int b) {
            return b == 0 ? a : a / b;
        }
        @Override public String simbolo() { return "/"; }
    };
}
```

Perche' questa forma: le slide la introducono con una motivazione di ingegneria del software, non di sintassi. Quando *ogni* bottone ha un comportamento diverso, avere una classe con nome per ciascuno provocherebbe una *"proliferation of standard classes, making the code not readable"*. Con le classi anonime *"the behaviour associated with a button is placed right where such button is instantiated"*.
Fonte: slide L04, pag. 47.

Wrong Concept 3.1: La classe anonima e' "un pezzo di codice passato come parametro"

Le slide insistono su questo punto: *"we get the impression that a piece of code could be the parameter of the call of the method* `addActionListener`. ***But it is not so**: it is a fully fledged (anonymous) class that is created, and an object of it is then instantiated and passed as a parameter"*.

E' la conferma della slide 4: Java non passa funzioni, passa **oggetti**. La sintassi concisa maschera un oggetto vero.

Fonte: slide L04, pag. 49.

3.3.6 Forma E — lambda

```
c.setOperazione((a, b) -> a > b ? a : b);   // lambda
c.setOperazione(Math::sqrt);                 // method reference
```

Possibile perche' `Operazione` ha un solo metodo astratto (*functional interface* de facto).

3.4 La misura decisiva: i nomi generati dal compilatore

Le cinque forme non sono varianti stilistiche: producono **artefatti diversi**. Chiedendo a runtime il nome della classe di ciascuna strategia, si ottiene (output reale):

```
A static nested -> NL1_Java_SimpleCalc$Somma
B inner         -> NL1_Java_SimpleCalc$MoltiplicaPerCampo
C local         -> NL1_Java_SimpleCalc$1SottrazioneLimitata
D anonima       -> NL1_Java_SimpleCalc$1
E lambda        -> NL1_Java_SimpleCalc$$Lambda/0x000000003f041000
```

E i file effettivamente prodotti da `javac`:

```
NL1_Java_SimpleCalc$1.class                     <- anonima
NL1_Java_SimpleCalc$1SottrazioneLimitata.class  <- local
NL1_Java_SimpleCalc$Calcolatrice.class
```

```
NL1_Java_SimpleCalc$MoltiplicaPerCampo.class     <- inner
NL1_Java_SimpleCalc$Operazione.class
NL1_Java_SimpleCalc$Somma.class                  <- nested
NL1_Java_SimpleCalc.class
```

Proposition 3.4.1 Cosa insegna questo elenco

- nested e inner producono un `.class` col **proprio nome** (`$Somma`, `$MoltiplicaPerCampo`): sono tipi a pieno titolo, distinguibili;
- la **local class** ha un **numero davanti al nome** (`$1SottrazioneLimitata`): il numero serve perche' due metodi diversi potrebbero dichiarare due local class omonime;
- la **anonima** ha **solo un numero** (`$1`): non ha un nome da conservare;
- la **lambda non ha alcun file** `.class`. Il suo nome contiene un indirizzo (`/0x...`) perche' la classe e' **generata a runtime** dalla JVM (`invokedynamic` + `LambdaMetafactory`).

Quindi una lambda **non e'** zucchero sintattico per una classe anonima: sono due meccanismi diversi con la stessa apparenza.

Misurato in `prove/strategy/NL1_Java_SimpleCalc.java` (compilato ed eseguito).

Chapter 4

C++: solo nested, piu' le lambda

4.1 L'equivalente dell'interfaccia: la classe astratta pura

In C++ non esiste `interface`: si usa una **classe astratta pura** — soli metodi virtuali puri, nessun dato di istanza — piu' un distruttore virtuale.

```
struct Bicycle {                        // "interfaccia" in C++
    virtual void changeCadence(int) = 0;
    virtual void changeGear(int) = 0;
    virtual void speedUp(int) = 0;
    virtual void applyBrakes(int) = 0;
    virtual ~Bicycle() = default;       // obbligatorio: si distrugge via base*
};
```

E l'equivalente di un **default method** e' un metodo virtuale **con corpo**. Ma qui la corrispondenza si rompe, e in modo istruttivo.

Wrong Concept 4.1: Un metodo virtuale con corpo e' l'equivalente di un default method

Non del tutto. Un default method Java non aggiunge ne' stato ne' un sotto-oggetto alla classe che implementa l'interfaccia. Un metodo virtuale C++ porta con se' una **base vera**, con vtable e sotto-oggetto — e quindi riapre il *diamond problem* che le interfacce Java evitano per costruzione.

4.1.1 I tre casi delle slide, tradotti

Caso	Java	C++
Pegasus (istanza vs default)	compila da solo: vince `Horse` (regola 1)	**ambiguo**: *"request for member 'identifyMyself' is ambiguous"*. Va risolto a mano
Dragon (antenato comune)	compila da solo: vince `EggLayer` (regola 2)	serve l'**ereditarieta' virtuale**; poi funziona per **dominanza**
FlyingCar (default indipendenti)	errore, override obbligatorio, `FlyCar.super.f()`	**identico**, ma la sintassi e' `FlyCar::f()`

Proposition 4.1.1 Il caso Dragon e' l'analogo esatto della regola 2

In C++ il caso `Dragon` si scrive cosi':

```
struct Animal       { virtual std::string identifyMyself() const
                      { return "I am an animal."; } };
struct EggLayer     : virtual Animal {          // <- virtual!
    std::string identifyMyself() const override
    { return "I am able to lay eggs."; } };
```

```
struct FireBreather : virtual Animal { };
struct Dragon : EggLayer, FireBreather { };     // nessun override
```

Funziona (output verificato: *"I am able to lay eggs."*) grazie alla **dominanza**: con `virtual` c'e' un solo sotto-oggetto `Animal`, ed `EggLayer` lo domina.

E' *la stessa idea* della regola 2 di Java ("i metodi gia' sovrascritti sono ignorati"), ottenuta pero' con un meccanismo del tutto diverso — e che in Java non serve, perche' le interfacce non portano sotto-oggetti da condividere.

Example 4.1.1 (La misura che spiega tutto: `sizeof`)

```
sizeof(Pegasus) = 24 byte     (tre basi polimorfiche -> TRE vptr)
```

In Java implementare dieci interfacce **non aggiunge nulla** all'oggetto: le interfacce non hanno stato di istanza. In C++ ogni base polimorfica costa un vptr. E' la conferma quantitativa della frase della slide 14 (*"the absence of instance data eliminates the ambiguities of, say, C++"*): non e' solo una questione di ambiguita' logica, e' anche una questione di **layout in memoria**.

Misurato in `prove/strategy/NL6_interfacce_altri_linguaggi.cpp` (compilato ed eseguito).

4.2 "In C++ this is just related to visibility and naming"

La frase della slide 6 e' esatta, e ha tre conseguenze precise, tutte verificate.

Proposition 4.2.1 Conseguenza 1: non esiste l'inner class

In C++ una classe annidata e' **sempre** l'equivalente di una `static` nested class di Java: **non** ha alcun legame implicito con un'istanza della classe esterna.

```
struct E {
    int v = 1;
    struct N {
        int f() { return v; }   // ERRORE: invalid use of non-static
    };                          //          data member 'E::v'
};
```

Per ottenere il comportamento di una inner class Java bisogna **passare esplicitamente** un puntatore all'oggetto esterno — che e' esattamente cio' che il compilatore Java genera di nascosto.

Proposition 4.2.2 Conseguenza 2: ma l'accesso ai privati c'e'

Dal C++11 una classe annidata ha accesso ai membri **privati** della classe che la racchiude (ha gli stessi diritti di un membro). Verificato: la nested `Somma` incrementa il `contatoreStatico_` privato dell'enclosing.

Proposition 4.2.3 Conseguenza 3: lo scope, invece, c'e' eccome

Il nome di una classe annidata e' **qualificato**: `Calcolatrice::Somma`. Non e' visibile come `Somma` nudo all'esterno. (Si vedra' nel capitolo sul C che questo, per le `struct`, e' una differenza C/C++ sostanziale.)

4.3 Le tre classi dello Strategy in C++

4.3.1 Strategy e Context, con le strategie annidate

```
// STRATEGY: in C++ non esiste 'interface' -> classe astratta pura
struct Operazione {
```

```
    virtual int applica(int a, int b) const = 0;
    virtual std::string simbolo() const { return "?"; }
    virtual ~Operazione() = default;      // sempre, se si distrugge via base*
};

class Calcolatrice {                      // CONTEXT
    int accumulatore_ = 0;
    const Operazione* operazione_ = nullptr;
    static int contatoreStatico_;         // membro STATIC privato
    int campoDiIstanza_ = 100;            // membro di ISTANZA privato

public:
    // --- FORMA A: nested class ---
    struct Somma : Operazione {
        int applica(int a, int b) const override {
            contatoreStatico_++;          // OK: privato static dell'enclosing
            // return a + b + campoDiIstanza_;  // ERRORE: nessuna istanza!
            return a + b;
        }
        std::string simbolo() const override { return "+"; }
    };
    // ...
};
int Calcolatrice::contatoreStatico_ = 0;   // definizione fuori dalla classe
```

4.3.2 Forma B — l'equivalente dell'inner class, a mano

```
class MoltiplicaPerCampo : public Operazione {
    const Calcolatrice* esterno_;         // <- l'"enclosing instance", A MANO
public:
    explicit MoltiplicaPerCampo(const Calcolatrice* e) : esterno_(e) {}
    int applica(int a, int b) const override {
        return a * b % esterno_->campoDiIstanza_;   // accede al privato
    }
};

// factory che imita la sintassi Java 'esterno.new Inner()'
std::unique_ptr<Operazione> creaMoltiplicaPerCampo() const {
    return std::make_unique<MoltiplicaPerCampo>(this);
}
```

Perche' questa struttura: rende *visibile nel codice* cio' che Java nasconde. Il costo dell'"inner class" Java c'e' anche in Java — un campo puntatore in piu' per oggetto — solo che non si vede. In C++ lo si paga esplicitamente, e infatti si misura:

```
sizeof(Calcolatrice::Somma)        = 8    (solo il vptr)
sizeof(MoltiplicaPerCampo)         = 16   (vptr + puntatore all'esterno)
```

Misurato in `prove/strategy/NL2_cpp_SimpleCalc.cpp` (compilato ed eseguito).

4.3.3 Forma C — local class, e i suoi due vincoli

Le slide dicono che "sia Java sia C++ permettono di definire una classe dentro il corpo di una funzione". E' vero, ma in C++ con due limiti che Java non ha, entrambi verificati con messaggi d'errore reali.

Wrong Concept 4.2: Una local class C++ puo' usare le variabili locali della funzione

No. Puo' usare solo variabili `static`, globali, enum e tipi. Le variabili **automatiche** sono vietate:

```
void g() {
    int automatica = 1;
    class L {
    public:
```

```
        int f() { return automatica; }   // ERRORE
    };
}
```

Messaggio reale del compilatore:

```
error: use of local variable with automatic storage
       from containing function
```

Il motivo: una local class e' un *tipo*, e i suoi metodi sono funzioni ordinarie; non esiste alcun meccanismo che copi le variabili locali dentro l'oggetto. In Java quel meccanismo esiste (la cattura per valore delle *effectively final*); in C++ e' stato aggiunto solo nel 2011, e si chiama **lambda**.

Wrong Concept 4.3: Una local class C++ puo' avere membri dati `static`

No. Messaggio reale:

```
error: local class 'class g()::L' shall not have
       static data member 'int g()::L::m'
```

Un membro static richiederebbe una definizione a livello di namespace, che per una classe locale non e' esprimibile.

Nell'esempio eseguibile, per aggirare il primo vincolo, il valore "catturato" viene passato tramite una variabile `static` locale — e il commento nel codice segnala che si tratta di un *workaround*, non della soluzione idiomatica:

```
std::unique_ptr<Operazione> creaSottrazioneLimitata(int minimo) {
    static int minimoStatico = 0;      // workaround: non 'minimo' diretto
    minimoStatico = minimo;

    class SottrazioneLimitata : public Operazione {
    public:
        int applica(int a, int b) const override {
            // return std::max(a - b, minimo);   // *** ERRORE ***
            int m = minimoStatico;               // ...quindi la statica
            return (a - b) > m ? (a - b) : m;
        }
    };
    return std::make_unique<SottrazioneLimitata>();
}
```

Note:

Questo workaround ha un difetto grave che vale la pena vedere: la variabile `static` e' **condivisa fra tutte** le chiamate. Creando due strategie con minimi diversi, la seconda *sovrascriverebbe* il valore visto dalla prima. E' precisamente il problema che le closure risolvono, e il motivo per cui in C++ moderno per questo scopo si usa una lambda, non una local class.

4.3.4 Forma D — la lambda: la vera classe locale del C++

```
int minimo = 5;                              // variabile AUTOMATICA
cf.setOperazione([minimo](int a, int b) {    // <- catturata!
    int r = a - b;
    return r > minimo ? r : minimo;
}, "-(>=5) lambda");
```

Proposition 4.3.1 Cos'e' davvero una lambda

Il compilatore genera un **tipo senza nome** (*closure type*), **locale** allo scope in cui la lambda appare,

con un `operator()` e un membro per ogni variabile catturata. E' quindi letteralmente una **classe locale anonima con cattura**: risolve in un colpo il limite della local class e fornisce l'equivalente della classe anonima di Java.

Fonte: Thinking in C++ *(Eckel), Vol. 2, cap. 10, "Strategy".*

Le dimensioni misurate rendono concreto il confronto:

Forma	**sizeof**	**Composizione**
nested class con un metodo virtuale	8	solo il vptr
"inner class" con puntatore all'esterno	16	vptr + puntatore
lambda che cattura un `int`	**4**	solo la cattura, **nessun vptr**
lambda senza cattura	**1**	oggetto vuoto
`std::function<int(int,int)>`	32	type erasure

Note:

Una lambda e' *piu' piccola* di una strategia con funzione virtuale, perche' non ha vptr: il tipo e' noto staticamente. Il prezzo e' che ogni lambda ha un **tipo diverso**, quindi per metterle in una variabile comune serve `std::function` — che reintroduce l'indirezione (e i 32 byte). E' lo stesso trade-off monomorfizzazione/erasure incontrato nei template.

Misurato in `prove/strategy/NL2_cpp_SimpleCalc.cpp` (compilato ed eseguito).

Chapter 5

Python: un solo costrutto, tre comportamenti

5.1 L'equivalente dell'interfaccia: tre possibilita'

Python offre **tre** modi di dichiarare il tipo di una strategia, che coprono uno spettro che in Java non esiste.

Meccanismo	Tipo di controllo	Quando fallisce
`abc.ABC` + `@abstractmethod`	**nominale**: serve ereditare esplicitamente	a **runtime**, all'istanziazione
`typing.Protocol`	**strutturale**: basta avere i metodi	a **compile time**, ma solo se lanci `mypy`
niente (duck typing)	nessuno	a runtime, alla prima chiamata mancante

```python
# 1. NOMINALE: l'equivalente piu' vicino a 'interface' di Java
class Bicycle(ABC):
    @abstractmethod
    def change_cadence(self, v: int) -> None: ...
    def descrivi(self) -> str:              # <- l'equivalente di un DEFAULT
        return f"{type(self).__name__} generica"

# 2. STRUTTURALE: nessuna ereditarieta' richiesta
@runtime_checkable
class Operazione(Protocol):
    def applica(self, a: int, b: int) -> int: ...

class Somma:                                # NON eredita da Operazione!
    def applica(self, a, b): return a + b
```

Verificato: `issubclass(Somma, Operazione)` restituisce `True` pur non essendoci alcuna parentela dichiarata — conta la *struttura*. Ed e' l'opposto esatto di Java, dove serve `implements`.
Misurato in `prove/strategy/NL7_interfacce_python.py` (compilato ed eseguito).

5.2 I tre casi delle slide, risolti dall'MRO

Python non ha ne' le due regole di Java ne' la dominanza del C++: ha l'**MRO** (Method Resolution Order, linearizzazione C3). I risultati spesso coincidono, ma il meccanismo e' diverso — e le differenze si vedono.

Caso	Risultato	Meccanismo
Pegasus	*"I am a horse."* (come Java)	MRO: `[Pegasus, Horse, Flyer, Mythical, object]` — vince il **primo nell'ordine**, non "la classe"
Dragon	*"I am able to lay eggs."* (come Java)	MRO: `[Dragon, EggLayer, FireBreather, Animal, object]` — C3 mette `EggLayer` prima di `Animal`
FlyingCar	**nessun errore!**	prende silenziosamente il primo dell'MRO

Wrong Concept 5.1: Python risolve i conflitti "come Java"

Coincidenza di risultati, non di meccanismo — e due differenze pesanti, entrambe verificate:

1. **l'ordine delle basi conta.** Scrivendo `class Pegasus(Flyer, Horse, Mythical)` invece di `class Pegasus(Horse, Flyer, Mythical)`, il risultato **cambia**: diventa *"I am able to fly."*. In Java la regola "istanza batte default" vale *indipendentemente* dall'ordine in cui si scrivono le interfacce;
2. **il conflitto vero non e' un errore.** Il caso `FlyingCar` — due implementazioni indipendenti dello stesso metodo — in Java **non compila**; in Python passa inosservato e vince il primo nell'MRO. E' il caso in cui Python e' piu' permissivo e *piu' pericoloso*.

Misurato in `prove/strategy/NL7_interfacce_python.py` (compilato ed eseguito).

Note:

Perche' in Python il problema dei *dati* duplicati non si pone mai: lo stato di un'istanza vive in **un solo** `__dict__`, qualunque sia la profondita' della gerarchia. Non esistono sotto-oggetti separati da condividere, quindi non serve nulla di analogo all'ereditarieta' virtuale del C++. Resta solo il problema dell'*ordine* di risoluzione — che e' appunto l'MRO.

5.3 Nested, inner, local: un solo costrutto

Python non ha alcuna distinzione *sintattica* fra nested, inner, local e anonymous class: ha un solo costrutto, `class`, che si comporta diversamente a seconda di **dove** lo si scrive. Le differenze vanno quindi misurate, non lette nella sintassi.

5.4 Misura 1: la classe annidata NON vede lo scope esterno

```
class Esterna:
    attributo = "ESTERNO"
    class Annidata:
        def prova(self):
            return attributo        # nome NUDO
```

Risultato reale: `NameError: name 'attributo' is not defined.`

Proposition 5.4.1 Perche'

Il corpo di una classe crea uno **namespace**, non uno *scope* annidato per la ricerca dei nomi: il corpo di `Annidata` non "vede" il corpo di `Esterna`. Bisogna qualificare: `Esterna.attributo`.

Il comportamento e' quindi lo stesso delle **nested class del C++**: puro **namespacing**. Python *non* ha inner class. Confermato da `__qualname__`, che vale `Esterna.Annidata`.

Misurato in `prove/strategy/NL3_python_SimpleCalc.py` (compilato ed eseguito).

Example 5.4.1 (Un'asimmetria che sorprende)

Dentro il corpo di una classe, *mentre viene definita*, i nomi precedenti **sono** visibili. Nel file eseguibile:

```
class Calcolatrice:
    class Operazione(ABC): ...
    class Somma(Operazione):        # <- 'Operazione' NUDO funziona qui!
        def applica(self, a, b):
            Calcolatrice.contatore_statico += 1   # <- ma questo va qualificato
            return a + b
```

La differenza: `Operazione` viene risolto *durante* l'esecuzione del corpo di `Calcolatrice` (quando il namespace locale della classe e' ancora attivo); `contatore_statico` invece viene risolto *dopo*, quando il metodo `applica` viene chiamato — e a quel punto quel namespace non esiste piu'.

5.5 Misura 2: la classe locale SI' cattura (a differenza del C++)

```
def fabbrica(moltiplicatore):
    class Locale:
        def calcola(self, x):
            return x * moltiplicatore    # variabile LOCALE della funzione
    return Locale
```

Risultati reali: `fabbrica(3)` produce una classe che moltiplica per 3, `fabbrica(5)` per 5.

Proposition 5.5.1 Il contrasto col C++

Questa e' esattamente la riga che in C++ produce *"use of local variable with automatic storage from containing function"*. In Python funziona perche' i metodi sono **closure**: catturano la *cella* della variabile nell'ambiente lessicale che le racchiude.

Python quindi **non ha** l'inner class di Java (misura 1), ma **ha** la local class con cattura — che in C++ richiede una lambda.

5.6 Misura 3: ogni chiamata crea una classe NUOVA

```
L3, L5 = fabbrica(3), fabbrica(5)
L3 is L5             -> False
isinstance(L3(), L5) -> False
stesso __qualname__  -> True  (fabbrica.<locals>.Locale)
```

Proposition 5.6.1 Conseguenza pratica pesante

Sono **due classi distinte** con lo *stesso nome sorgente*. Un'istanza della prima **non** e' istanza della seconda: quindi `isinstance` non e' affidabile per confrontare oggetti prodotti da chiamate diverse della stessa factory.

In Java la local class e' compilata **una volta sola** (`$1SottrazioneLimitata`) e le sue istanze condividono il tipo. In Python l'oggetto-classe e' **ricreato a ogni chiamata**, con il costo di memoria e tempo che ne segue — ragione per cui, se non serve la cattura per-chiamata, conviene definire la classe fuori.

Misurato in `prove/strategy/NL3_python_SimpleCalc.py` (compilato ed eseguito).

5.7 Le tre classi dello Strategy in Python

```
class Calcolatrice:                      # CONTEXT
    contatore_statico = 0                # attributo di CLASSE

    class Operazione(ABC):               # STRATEGY, annidata
        @abstractmethod
        def applica(self, a, b): ...
```

```
        def simbolo(self): return "?"

    class Somma(Operazione):              # FORMA A: annidata
        def applica(self, a, b):
            Calcolatrice.contatore_statico += 1     # va QUALIFICATO
            return a + b
        def simbolo(self): return "+"

    class MoltiplicaPerCampo(Operazione): # FORMA B: "inner", a mano
        def __init__(self, esterno):
            self._esterno = esterno       # enclosing instance ESPLICITA
        def applica(self, a, b):
            return a * b % self._esterno.campo

    def crea_moltiplica_per_campo(self):  # imita esterno.new Inner()
        return Calcolatrice.MoltiplicaPerCampo(self)
```

```
# FORMA C: local class, con cattura vera
def crea_sottrazione_limitata(minimo):
    class SottrazioneLimitata(Calcolatrice.Operazione):
        def applica(self, a, b):
            return max(a - b, minimo)     # <- cattura; in C++ e' errore
        def simbolo(self): return f"-(>={minimo})"
    return SottrazioneLimitata()

# FORMA D: la classe davvero "anonima", creata con type()
def divisione_sicura():
    return type(
        "",                                       # nome VUOTO
        (Calcolatrice.Operazione,),
        {"applica": lambda self, a, b: a if b == 0 else a // b,
         "simbolo": lambda self: "/"},
    )()
```

Verificato: `type(anon).__name__` e' la stringa vuota, e `isinstance(anon, Calcolatrice.Operazione)` e' `True` — e' una sottoclasse a tutti gli effetti, senza nome.

Note:

In Python la forma idiomatica per lo Strategy non e' nessuna di queste quattro: e' passare direttamente una **funzione**. Le classi servono quando la strategia ha stato o deve esporre piu' metodi. Il documento le mostra tutte perche' l'obiettivo qui e' confrontare l'*infrastruttura*, non scegliere lo stile.

Chapter 6

C: senza classi, senza scope, senza closure

Il C non ha classi. Le tre categorie delle slide vanno quindi tradotte in meccanismi diversi — e nel farlo emergono due limiti che gli altri tre linguaggi non hanno.

6.1 L'equivalente dell'interfaccia: la struct di puntatori

In C l'"interfaccia" e' una `struct` i cui campi sono **puntatori a funzione**: e' letteralmente la vtable, scritta a mano.

```c
typedef struct {                    /* l'"interfaccia" */
    int (*applica)(const struct Operazione *self, int a, int b);
    const char *simbolo;
} OperazioneVT;

struct Operazione {                 /* la "classe astratta" */
    const OperazioneVT *vptr;
};
```

Proposition 6.1.1 Cosa manca, rispetto a un'interfaccia vera

- **nessun controllo di completezza**: se dimentichi di assegnare un puntatore, non c'e' alcun errore di compilazione — solo un `NULL` che a runtime fa crashare. In Java il compilatore *rifiuta* una classe che non implementa tutti i metodi;
- **nessun default method**: si puo' lasciare un campo a `NULL` e far controllare al Context (`if (vt->f) vt->f(...)`) — ed e' esattamente cio' che fa il VFS del kernel Linux con `struct file_operations`;
- **nessun conflitto possibile**: non esistendo ereditarieta' multipla, il problema che le due regole di Java risolvono *non si presenta*. Se servono due "interfacce", si mettono due vtable nella struct e si sceglie a mano quale usare.

Misurato in `prove/strategy/NL4_c_SimpleCalc.c` (compilato ed eseguito).

6.2 Scoperta 1: annidare una `struct` NON crea uno scope

```c
struct Contenitore {
    int campo;
    struct Annidata {        /* "annidata" solo tipograficamente */
        int x;
    } ann;
};

/* Questa riga COMPILA in C: */
struct Annidata annidata_usata_fuori = { 42 };
```

Verificato: il programma compila con `gcc -Wall -Wextra` e stampa `42`.

Proposition 6.2.1 La differenza C / C++

In **C** annidare una `struct` e' puramente tipografico: il tag `Annidata` finisce nello **stesso spazio dei nomi globale** del tag esterno. Non c'e' alcun incapsulamento.

In **C++** la stessa riga e' un **errore**:

```
error: variable 'Annidata usata_fuori' has initializer
       but incomplete type
```

perche' li' il nome corretto sarebbe `Contenitore::Annidata`. E' un caso in cui il C++ *non* e' un soprainsieme del C, e mostra che quel poco che le slide attribuiscono al C++ ("visibility and naming") e' comunque piu' di quanto offra il C.

Misurato in `prove/strategy/NL4_c_SimpleCalc.c` (compilato ed eseguito).

6.3 Scoperta 2: le funzioni locali non esistono in C standard

```c
int main(void) {
    int fattore = 3;
    int moltiplica(int x) { return x * fattore; }   /* ??? */
    return moltiplica(10);
}
```

Compilato con `-std=c11 -pedantic`:

```
warning: ISO C forbids nested functions [-Wpedantic]
```

Funziona solo come **estensione GNU** (`-std=gnu11`).

Proposition 6.3.1 La conseguenza per lo Strategy

Si puo' definire una `struct` dentro una funzione (e quella e' davvero locale), ma **non le si puo' dare un metodo**: le funzioni che implementano il comportamento devono stare a livello globale. Quindi:

- la **vtable** e le funzioni-strategia sono sempre globali;
- la "localita"' puo' riguardare solo i *dati*;
- non esistendo closure, il contesto va passato **a mano**.

Le quattro forme di Java collassano in **una sola** tecnica.

6.4 Le tre classi dello Strategy in C

6.4.1 Strategy: una struct di puntatori a funzione

```c
struct Operazione;                      /* dichiarazione anticipata */

typedef struct {                        /* la "vtable" a mano */
    int (*applica)(const struct Operazione *self, int a, int b);
    const char *simbolo;
} OperazioneVT;

struct Operazione {                     /* la "classe base astratta" */
    const OperazioneVT *vptr;
};
```

6.4.2 Context e delega

```c
typedef struct {
    int accumulatore;
    const struct Operazione *operazione;
    int campo;                      /* lo "stato dell'oggetto esterno" */
} Calcolatrice;

static void calc_premi_numero(Calcolatrice *c, int n) {
    if (c->operazione == NULL) { c->accumulatore = n; return; }
    /* la delega, identica a strategy.algorithm() di Java/C++ */
    c->accumulatore = c->operazione->vptr->applica(c->operazione,
                                                   c->accumulatore, n);
}
```

6.4.3 Le tre forme, tradotte

```c
/* FORMA A -- l'equivalente della NESTED: nessuno stato proprio */
typedef struct { struct Operazione base; } Somma;

static int somma_applica(const struct Operazione *self, int a, int b) {
    (void)self;                     /* non serve alcuno stato */
    contatore_statico++;            /* usa il "membro static" globale */
    return a + b;
}
static const OperazioneVT SOMMA_VT = { somma_applica, "+" };

/* FORMA B -- l'equivalente dell'INNER: porta il puntatore all'esterno */
typedef struct {
    struct Operazione base;
    const Calcolatrice *esterno;    /* l'"enclosing instance", A MANO */
} MoltiplicaPerCampo;

static int moltiplica_applica(const struct Operazione *self, int a, int b) {
    const MoltiplicaPerCampo *m = (const MoltiplicaPerCampo *)self;
    return a * b % m->esterno->campo;       /* legge lo stato dell'esterno */
}

/* FORMA C -- l'equivalente della LOCAL: il valore "catturato" e' un campo */
typedef struct {
    struct Operazione base;
    int minimo;                     /* la "cattura", esplicita */
} SottrazioneLimitata;
```

Proposition 6.4.1 Il ruolo del "primo membro"

In tutte e tre le forme, `struct Operazione base;` e' il **primo** membro. Lo standard C garantisce l'assenza di padding all'inizio di una struct, quindi il cast fra `struct Operazione*` e il tipo concreto e' un **no-op**: e' cio' che permette al Context di trattare uniformemente strategie diverse.

6.4.4 Forma D — puntatore a funzione e contesto `void*`

```c
typedef int (*FunzioneBinaria)(void *ctx, int a, int b);

typedef struct { int soglia; } CtxMassimo;

static int massimo_con_soglia(void *ctx, int a, int b) {
    CtxMassimo *c = (CtxMassimo *)ctx;
    int m = a > b ? a : b;
    return m > c->soglia ? m : c->soglia;
}

static int applica_funzione(FunzioneBinaria f, void *ctx, int a, int b) {
```

```
    return f(ctx, a, b);            /* la delega, senza oggetti */
}
```

E' il pattern di `qsort_r`, dei callback POSIX, di GTK e del VFS del kernel Linux: la coppia (*puntatore a funzione*, *void**) e' l'equivalente C di una closure.
Misurato in `prove/strategy/NL4_c_SimpleCalc.c` (compilato ed eseguito).

Chapter 7

Confronto finale

7.1 La stessa strategia, quattro infrastrutture

Tutti e quattro i programmi calcolano `20` ⟨`op`⟩ `6`. Output reali affiancati:

Forma	Java	C++ / Python	C
A "nested"	`static class`	`struct` annidata / `class` in `class`	`struct` + vtable
B "inner"	**automatico** (non-static)	puntatore all'esterno **a mano**	campo `esterno` **a mano**
C "local"	con cattura	C++: **senza** cattura; Python: **con**	solo dati, metodi globali
D "anonima"	`new I(){...}`	lambda / `type("")`	*non esiste*

7.2 Il TIPO della strategia: le quattro dichiarazioni

	Come si dichiara	Conflitti fra piu' basi	Costo per l'oggetto
Java	`interface` + `default`/`static`	le **due regole**; il conflitto vero e' un **errore**	**nessuno**: niente stato di istanza
C++	classe astratta pura + dtor virtuale	**ambiguo**: risolvi a mano, o ereditarieta' **virtuale** + dominanza	**un vptr per base** (Pegasus: 24 byte)
Python	`ABC` (nominale) o `Protocol` (strutturale)	**MRO**: nessun errore, vince il primo	nessuno: un solo `__dict__`
C	`struct` di puntatori a funzione	*non esistono*: niente ereditarieta'	un puntatore (il vptr)

Proposition 7.2.1 La differenza piu' importante fra i quattro

Solo **Java** tratta il conflitto fra comportamenti ereditati come un **errore di compilazione** che obbliga il programmatore a decidere. C++ lo segnala come ambiguita' (ma solo al punto d'uso, o alla definizione della classe se manca il final overrider); Python **non lo segnala affatto** e sceglie in silenzio; il C non puo' nemmeno averlo.

7.3 Le quattro affermazioni da ricordare

Proposition 7.3.1 Sintesi

1. Solo **Java** ha il legame *implicito* con l'istanza esterna (inner class). In C++, Python e C va scritto a mano — e in Java quel campo puntatore esiste comunque, solo che non si vede.
2. Solo **C++** vieta alla classe locale di leggere le variabili automatiche. Per questo la lambda, nel C++

moderno, e' la vera "classe locale".

3. Solo **Python** ricrea l'oggetto-classe a ogni chiamata della funzione che la contiene, con la conseguenza che `isinstance` fra classi omonime da chiamate diverse e' `False`.

4. Solo il **C** non ottiene *alcun* incapsulamento annidando: il tag resta globale, e senza funzioni locali tutte le forme collassano in "struct + puntatori a funzione + contesto esplicito".

7.4 Come scegliere, in pratica

Se la strategia...	**usa...**
non ha bisogno dello stato del Context	**nested/static**: e' la forma di default, non trattiene l'oggetto esterno
deve leggere lo stato *vivo* del Context	Java: **inner class**; altrove: puntatore/riferimento esplicito
serve solo dentro un metodo, e cattura un parametro	Java: **local class**; C++: **lambda**; Python: **local class**; C: campo nella struct
e' usata una volta sola, in un'espressione	**anonima** (Java) / **lambda** (C++, Python)
e' un semplice calcolo senza stato	**lambda** o funzione libera; in C un puntatore a funzione nudo

Chapter 8

Esecuzione e verifica

8.1 I quattro programmi

File	Comando
`NL1_Java_SimpleCalc.java`	`javac NL1_Java_SimpleCalc.java && java -cp . NL1_Java_SimpleCalc`
`NL2_cpp_SimpleCalc.cpp`	`g++ -std=c++17 -Wall -Wextra NL2_cpp_SimpleCalc.cpp -o NL2 && ./NL2`
`NL3_python_SimpleCalc.py`	`python3 NL3_python_SimpleCalc.py`
`NL4_c_SimpleCalc.c`	`gcc -std=c11 -Wall -Wextra NL4_c_SimpleCalc.c -o NL4 && ./NL4`
`NL5_Java_Interfacce.java`	`javac NL5_Java_Interfacce.java && java -cp . NL5_Java_Interfacce`
`NL6_interfacce_ altri_linguaggi.cpp`	`g++ -std=c++17 -Wall -Wextra NL6_*.cpp -o NL6 && ./NL6`
`NL7_interfacce_python.py`	`python3 NL7_interfacce_python.py`

8.2 Esperimenti: rompili apposta

Note:

Ogni "ERRORE" commentato nei sorgenti e' reale: scommentalo e leggi il messaggio del compilatore. Sono la parte piu' istruttiva.

1. **Java** — crea `new MoltiplicaPerCampo()` da `main` (che e' `static`): il compilatore chiede un'enclosing instance.
2. **Java** — riassegna la variabile catturata dalla local class dopo averla usata: *"local variables referenced from an inner class must be final or effectively final"*.
3. **C++** — nella local class, sostituisci `minimoStatico` con `minimo`: *"use of local variable with automatic storage from containing function"*.
4. **C++** — aggiungi `static int m;` alla local class: *"local class shall not have static data member"*.
5. **C++** — nella nested `Somma`, prova a leggere `campoDiIstanza_`: *"invalid use of non-static data member"* — la prova che non esiste l'inner class.
6. **Python** — nella classe annidata, usa `contatore_statico` senza qualificarlo: `NameError` a runtime (non a "compile time": Python non ha modo di accorgersene prima).
7. **C** — compila `NL4` con `-std=c11 -pedantic` dopo aver aggiunto una funzione dentro una funzione: *"ISO C forbids nested functions"*.
8. **C** — rinomina la `struct Annidata` in `Contenitore` e osserva il conflitto: la prova che i tag stanno tutti nello stesso spazio globale.

9. **Java** — togli l'override di `startEngine` da `FlyingCar`: *"inherits unrelated defaults"*.
10. **Java** — prova a implementare `Bicycle` (4 metodi astratti) con una lambda: *"is not a functional interface"*.
11. **C++** — togli l'override in `Pegasus`: *"request for member 'identifyMyself' is ambiguous"* — il caso che in Java compila da solo.
12. **C++** — togli `virtual` dalle basi di `EggLayer`/`FireBreather`: il caso `Dragon` diventa ambiguo.
13. **Python** — inverti l'ordine delle basi di `Pegasus`: il risultato **cambia** (in Java no).

Chapter 9

Fonti

- Slide del corso: *L04 — Focus on the strategy pattern in Java* (Giancarlo Succi, Universita' di Bologna), 51 pagine — in particolare le pagg. 6–12 (nested / inner / local class), **13–32 (interfacce, default e static method, le due regole di risoluzione dei conflitti, gli esempi Pegasus / Dragon / FlyingCar / Mustang / DoIt)**, 33–36 (anonymous class e listener), 37–50 (`SimpleCalc` con le tre implementazioni di `ActionListener`).
- Bruce Eckel, *Thinking in C++*, Vol. 2, cap. 10 (*Design Patterns*), sezione *"Strategy: choosing the algorithm at runtime"*, e cap. 5 per il policy-based design: https://web.mit.edu/merolish/ticpp/TicV2.html#_Toc53985877
- Wikipedia, *Strategy pattern* — per la tassonomia dei quattro meccanismi (puntatori a funzione, first-class function, classi/istanze, reflection): https://en.wikipedia.org/wiki/Strategy_pattern
- Cartella `prove/strategy/` — i quattro programmi `NL1`–`NL4` e i test di misura, tutti compilati ed eseguiti su questa macchina. **Ogni output, messaggio d'errore e valore di `sizeof` citato in questo documento proviene da un'esecuzione reale**, non da memoria.